❷ The number of replicas

NOTE In a real scenario, you would probably want Kubernetes to dynamically scale applications in and out depending on the current workload, rather than providing a fixed number. Dynamic scaling is a pivotal feature of any cloud platform. In Kubernetes, it's implemented by a dedicated component called Horizontal Pod Autoscaler based on well-defined metrics, such as the CPU consumption per container. For more information, refer to the Kubernetes documentation (https://kubernetes.io/docs).

The next section will cover configuring CPU and memory for Spring Boot containers running in Kubernetes.

15.2.2 Configuring CPU and memory for Spring Boot containers

When dealing with containerized applications, it's best to assign *resource limits* explicitly. In chapter 1 you learned that containers are isolated contexts leveraging Linux features, like namespaces and cgroups, to partition and limit resources among processes. However, suppose you don't specify any resource limits. In that case, each container will have access to the whole CPU set and memory available on the host machine, with the risk of some of them taking up more resources than they should and causing other containers to crash due to a lack of resources.

For JVM-based applications like Spring Boot, defining CPU and memory limits is even more critical because they will be used to properly size items like JVM thread pools, heap memory, and non-heap memory. Configuring those values has always been a challenge for Java developers, and it's critical since they directly affect application performance. Fortunately, if you use the Paketo implementation of Cloud Native Buildpacks included in Spring Boot, you don't need to worry about that. When you packaged the Catalog Service application with Paketo in chapter 6, a *Java Memory Calculator* component was included automatically. When you run the